

Introduction to HTML

Elements

An element is a piece of content in an HTML document. It's usually denoted by a pair of tags like `<p>` and `</p>`. Some elements don't need a closing tag. These are called *void elements*. For example:

- Images `<img>`
- Meta tags `<meta>`
- Line breaks `<br>`

Attributes

Attributes provide extra information to the browser, such as:

- Where a link should go: `<a href="...">`
- The source of an image and its description ``

The two most common attributes are `id` (to uniquely identify an element on the page) and `class` (for grouping and styling, which we'll see later). Those two can be applied to any element, and can also be used together if needed.

Relative vs. Absolute URLs

A URL tells the browser where to find a resource. **Relative URLs** point to a file relative to the current page. These are common for linking to files within the same project. For example:

- In the same directory as the HTML page: ``
- In a subdirectory: ``
- In a parent directory: ``

Absolute URLs include the full web address (protocol, domain, and path). These are needed for resources hosted on other websites. For example: ``

There is a special case for linking to the same page. Instead of using a relative URL, you can use a special anchor link like `<a href="#section">`. This will scroll to whatever element has the `id="section"` attribute applied to it.

Nesting

You can put elements inside other elements. For example:

- Links inside of paragraphs: `<p><a href="...">...</a></p>`
- Images inside links: `<a href="..."></a>`
- List items inside of lists (see next section)

Lists

Lists group related items. You can make **unordered lists** using `<ul>` or **ordered lists** using `<ol>`.

```
<ul>
<li>Apples</li>
<li>Oranges</li>
<li>Bananas</li>
</ul>
```



- Apples
- Oranges
- Bananas

You can also nest lists inside each other for multiple levels:

```
<ol>
  <li>Morning</li>
  <li>Afternoon
    <ul>
      <li>Lunch</li>
      <li>Nap</li>
    </ul>
  </li>
  <li>Evening</li>
</ol>
```



1. Morning
2. Afternoon
 - Lunch
 - Nap
3. Evening

Boilerplate

Every HTML document should have the following structure:

```
<!DOCTYPE html>
<html>
<head>
```

`<!-- Tags that describe or affect the page but are not rendered directly -->`

```
<!-- Metadata, page title, icons, stylesheets, scripts, etc. -->

</head>
<body>

  <!-- The body contains the page's visible content -->

</body>
</html>
```

Inspecting Elements

Every modern browser has developer tools that let you look “under the hood.” You can right-click on any part of a page and choose **Inspect** (or press `F12` / `Ctrl+Shift+I` / `Cmd+Opt+I`). You'll immediately see:

- The page's *HTML structure* in a tree view.
- The *CSS rules* that apply to the selected element.
- Attributes and computed styles (fonts, colors, spacing).

This is useful for debugging (checking if your HTML/CSS is actually applied), learning (seeing how other sites are built), and quickly experimenting/iterating on your own code since editing directly in the inspector allows you to see changes instantly.

Try it out! Open a page, right-click a heading, and change the text in the inspector. It updates immediately! This further illustrates that HTML is just text interpreted by the browser.

Block vs. Inline Elements

Block elements take up the full width of the page. `<section>`, `<p>`, `<h1>`, `<div>`

Inline elements only take up as much space as they need, and are often used inside block elements. `<strong>`, `<em>`, `<span>`

- **Failure!** Your response was not saved.
- It's *really* nice weather outside today.

Grouping Content

`<div>` and `<span>` don't carry meaning, but they're useful for grouping and applying styles.

- `<div>` is block-level: it groups chunks of content.
- `<span>` is inline: it groups text inside a sentence.

Semantic HTML

`<div>` and `<span>` are the bread and butter of HTML. You *can* make any website using only those and text... but it's not best practice. Instead, we try to use semantic tags to describe the role and meaning of the content. `<header>` is more meaningful than `<div id="header">`, for example. This helps screen readers, search engines, and other tools understand your page beyond its appearance.

Here are some starter rules and an example:

- `<header>` : Intro content at the top of a page or section (logo, title, intro).
- `<nav>` : A set of major navigation links (site or section navigation).
- `<main>` : The unique primary content of the page (use once).
- `<section>` : A thematically related group of content that typically has a heading.
- `<article>` : Self-contained content that could stand alone or be syndicated (blog post, news item, forum post).
- `<aside>` : Tangential content (related links, ads, side notes).
- `<footer>` : Closing info for a page or section (author, copyright, related links).

```
<header>
  <h1>Site Title</h1>
  <nav>
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">Blog</a></li>
      <li><a href="#">About</a></li>
    </ul>
  </nav>
</header>

<main>
  <article>
    <h2>Post Title</h2>
    <p>Body text...</p>
  </article>

  <section>
    <h2>Related Resources</h2>
    <ul>
      <li><a href="#">Link A</a></li>
      <li><a href="#">Link B</a></li>
    </ul>
  </section>
```

```

</main>

<aside>
  <h2>Sidebar</h2>
  <p>Newsletter signup, ads, etc.</p>
</aside>

<footer>
  <p>© 2025 Your Name</p>
</footer>

```

This is one of those things that's more about *good judgement* rather than *hard and fast rules*. Don't sweat it too much! Here are a few tips to start with:

- **Use semantic elements wherever they clearly fit.** If unsure, fall back to a `<div>` with a clear class name.
- **Be consistent.** A consistent structure is better than "perfect" semantics used inconsistently.
- After you get your page working, do a quick semantic review:
 - "Can any `<div>` become `<section>` or `<article>`?"
 - "Does each `<section>` or `<article>` have a heading (`<h1>` through `<h6>`)?"

Tables

Tables are for **tabular data** (rows/columns). They are *not* for page layout.

Basic Table

```

<table>
  <tr><td>A1</td><td>B1</td></tr>
  <tr><td>A2</td><td>B2</td></tr>
</table>

```



A1	B1
A2	B2

Table with Headers

```

<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr><td>Alice</td><td>30</td></tr>
  <tr><td>Bob</td><td>25</td></tr>
</table>

```



Name	Age
Alice	30
Bob	25

Grouping Sections

```
<table>
  <thead>
    <tr><th>Product</th><th>Price</th></tr>
  </thead>
  <tbody>
    <tr><td>Apples</td><td>$2.00</td></tr>
    <tr><td>Oranges</td><td>$3.00</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total</td><td>$5.00</td></tr>
  </tfoot>
</table>
```



Product	Price
Apples	\$2.00
Oranges	\$3.00
Total	\$5.00

Merging Cells

```
<table>
  <tr>
    <th colspan="2">Schedule</th>
  </tr>
  <tr>
    <td>Morning</td><td>Work</td>
  </tr>
  <tr>
    <td rowspan="2">Afternoon</td><td>Study</td>
  </tr>
  <tr>
    <td>Exercise</td>
  </tr>
</table>
```



Schedule	
Morning	Work
Afternoon	Study
	Exercise

Accessibility with `scope`

```
<table>
  <tr>
    <th scope="col">Item</th>
    <th scope="col">Jul</th>
    <th scope="col">Aug</th>
    <th scope="col">Sep</th>
  </tr>
  <tr>
    <th scope="row">Apples</th>
    <td>$2.00</td>
```

Item	Jul	Aug	Sep
Apples	\$2.00		

```

    <td>$2.50</td>
    <td>$3.00</td>
  </tr>
  <tr>
    <th scope="row">Oranges</th>
    <td>$3.00</td>
    <td>$3.50</td>
    <td>$4.00</td>
  </tr>
</table>

```



Apples	\$2.00	\$2.50	\$3.00
Oranges	\$3.00	\$3.50	\$4.00

Forms

Forms let users enter and submit information. They are made up of controls like text fields, checkboxes, radio buttons, and buttons. A form usually sends its data to a server for processing.

```

<form>
  <input type="text">
  <button type="submit">Submit</button>
</form>

```



Other Input Types

```

<form>
  <input type="email">
  <input type="password">
  <input type="number">
  <input type="date">
  <input type="time">
  <input type="datetime-local">
  <input type="file">
  <input type="range">
  <input type="color">
  <input type="checkbox">
  <input type="radio">
</form>

```



 No file chosen

☐ ☐ ☐ Yes ☐ No

The examples above are simplified for brevity. With real forms, you should always use `<label>` elements so inputs are accessible and can be clicked on by screen readers. There are two ways to associate a label with an input, but the best practice is to use the `for` attribute on the label with a matching `id` attribute on the input.

Additionally, always add a `name` attribute to inputs so the data can be processed correctly by the server.

```
<form>
  <label for="name">Name</label>
  <input type="text" id="name" name="name">
  <button type="submit">Submit</button>
</form>
```

Reference Links

- [The World-Wide Web Project](#) (1991) - Launched by Tim Berners-Lee at CERN as the first web page explaining the World-Wide Web project
- [Space Jam](#) (1996) - A rare case of a preserved mid-90s studio website
- [Dole/Kemp '96](#) (1996) - One of the first U.S. presidential campaign websites
- [Arngren](#) (1997) - Norwegian collage-style store which became notorious in early web-design circles for its overwhelming collage style
- [ISO 8879:1986](#) (1986) - The first international standard for HTML's ancestor, SGML (Standard Generalized Markup Language)
- [MDN's Anatomy of an HTML Element](#)
- [MDN's HTML Elements Reference](#)